

POTION PANIC

(อลวนคนปรุงยา)

จัดทำโดย

นายปภาวิน โกการัตน์ รหัสนักศึกษา 6804062612366

ตอนเรียนที่ 3

เสนอ

ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 040613204 การเขียนโปรแกรมเชิงวัตถุ

(Object-Oriented Programming)

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เกี่ยวกับโครงงาน

ชื่อโครงงาน: POTION PANIC (อลวนคนปรุงยา)

นำเสนอโดย: นายปภาวิน โกการัตน์ (รหัสนักศึกษา 6804062612366 ตอนเรียนที่ 3)

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

Source Code: https://github.com/s6804062612366-web/POTION-PANIC-GAME

บทที่ 1 บทนำ

ที่มาและความสำคัญของโครงงาน

จากการเรียนรู้วิชาการเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming: OOP) ด้วยภาษา Java มุ่งเน้นให้ผู้เรียนเข้าใจหลักการพื้นฐาน เช่น การออกแบบคลาส (Class), ออบเจกต์ (Object), การห่อหุ้มข้อมูล (Encapsulation), การสืบทอดคุณสมบัติ (Inheritance) และการพ้องรูป (Polymorphism) ซึ่งในการวัดผลและฝึกปฏิบัติจริง รายวิชาได้กำหนดให้ผู้เรียนจัดทำโครงงานพัฒนาเกมคอมพิวเตอร์ด้วยภาษา Java เพื่อนำความรู้ ทฤษฎี และแนวคิดที่ได้เรียนรู้จากในชั้นเรียนมาประยุกต์ใช้สร้างสรรค์เป็นชิ้นงานที่สามารถทำงานได้จริง

เนื่องจากระบบของเกมเป็นรูปแบบโปรแกรมที่มีการโต้ตอบกับผู้เล่นอยู่ตลอดเวลา และจำเป็นต้องจำลององค์ประกอบที่หลากหลายทำงานร่วมกัน ซึ่งแต่ละองค์ประกอบต่างมีสถานะ (State) และพฤติกรรม (Behavior) เฉพาะตัว เช่น การเคลื่อนที่ การโจมตี หรือการตอบสนองต่อเงื่อนไขต่าง ๆ การนำหลักการเขียนโปรแกรมเชิงวัตถุมาประยุกต์ใช้ จึงเป็นหัวใจสำคัญที่ช่วยให้การจัดโครงสร้างโค้ดมีความเป็นระเบียบ สามารถแบ่งหน้าที่และความรับผิดชอบของแต่ละส่วนได้อย่างชัดเจน ทำให้ง่ายต่อการพัฒนาและปรับปรุงแก้ไขระบบ

ด้วยเหตุนี้ โครงงานเกม **"POTION PANIC (อลวนคนปรุงยา)"** จึงถูกพัฒนาขึ้นด้วยภาษา Java การพัฒนาโครงงานนี้นอกจากจะทำให้ผู้จัดทำได้ฝึกฝนทักษะการเขียนโปรแกรมและการแก้ปัญหาเชิงตรรกะจากการลงมือพัฒนาจริงแล้ว ยังได้สร้างสรรค์ผลงานเกมที่มอบทั้งความสนุกสนานและฝึกทักษะการตัดสินใจได้อย่างเพลิดเพลินอีกด้วย

ประโยชน์

1 ฝึกทักษะการแยกประสาท และการทำงานหลายอย่างพร้อมกัน (Multitasking)

2 พัฒนาทักษะการคิดวิเคราะห์ และการวางแผน (Strategic Thinking)

3 ฝึกความจำ และความรวดเร็วในการตอบสนอง (Memory & Reflexes)

4 ให้ความบันเทิง และคลายเครียด

5 เพื่อนำแนวคิด การเขียนโปรแกรมแบบเชิงวัตถุ (Object-Oriented Programming) มาประยุกต์ใช้ในการสร้างระบบงานที่ทำงานได้จริง

ขอบเขตของโครงงาน (Proposal)

POTION PANIC

(อลวนคนปรุงยา)

เนื้อเรื่อง

ในเกม POTION PANIC (อลวนคนปรุงยา) คุณจะได้รับบทเป็น **"ศิษย์ฝึกหัดนักปรุงยา"** ที่เพิ่งเข้ามารับเวรกะดึกเฝ้าหอคอยเพียงลำพัง แต่จู่ๆ คืนที่ควรจะเงียบสงบก็พังทลายลง เมื่อรอยแยกมิติเปิดออกและฝูงมอนสเตอร์กลายพันธุ์กำลังแห่กันมาที่หอคอย! ซ้ำร้าย **"สายธารวัตถุดิบเวทมนตร์"** ที่เชื่อมต่อกับใต้ดินดันทำงานรวน มันสูบเอาสมุนไพรและแร่ธาตุต่างๆ ไหลทะลักขึ้นมาอย่างบ้าคลั่งและไม่ขาดสาย คุณไม่มีกองทหาร ไม่มีดาบวิเศษ มีเพียงหม้อต้มยาเก่าๆ กับความจำในสมอง คุณต้องตั้งสติรับมือกับความอลวนนี้ คว้าวัตถุดิบที่ไหลมาตามสายพาน โยนลงหม้อให้ถูกสูตร แล้วสาดขวดยาเวทมนตร์สกัดกั้นพวกมอนสเตอร์ให้ทันเวลา ก่อนที่หอคอยแห่งนี้จะแตก!

วิธีการเล่น

กดปุ่มตัวเลข 1, 2, 3, 4, 5 เพื่อดึงวัตถุดิบจากสายพานในแต่ละช่องลงหม้อต้ม กดปุ่ม Spacebar เพื่อทำการผสมยา และใช้เมาส์คลิกซ้ายเพื่อเลือกขวดยาพร้อมกับคลิกเป้าหมายเพื่อปาโจมตีมอนสเตอร์

Storyboard

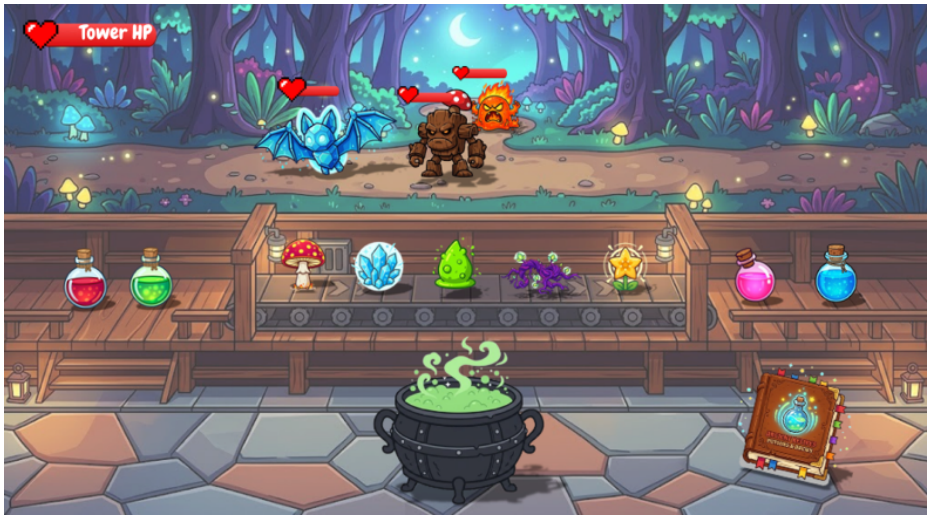
<u>ตัวละครและไอเทม</u>

			
มอนสเตอร์	**หม้อปรุงยา**	**วัตถุดิบปรุงยา**	**ขวดยา**

ฉาก



- **หน้าเริ่มเกม (Main Menu):** หน้าจอเริ่มต้นของเกมที่แสดงป้ายเมนู 2 ปุ่มหลัก ได้แก่ ปุ่ม "เริ่มเกม" สำหรับเข้าสู่เกม และปุ่ม "วิธีเล่น" สำหรับดูคู่มือปุ่มลัดและสูตรยาเบื้องต้น



- **หน้าระหว่างเกม (Gameplay):** หน้าจอหลักขณะเล่นที่แบ่งสัดส่วนการควบคุมเป็น 3 โซน ส่วนบนคือเส้นทางที่มอนสเตอร์เดินบุกเข้ามาพร้อมหลอดพลังชีวิต (HP) ส่วนกลางคือสายพานลำเลียงวัตถุดิบ 5 ช่องที่ไหลมา และส่วนล่างคือพื้นที่ของนักปรุงยาที่มีหม้อต้มสำหรับผสมไอเทม พื้นที่พักขวดยาพร้อมใช้ และสมุดสูตรยาสำหรับดูสูตรการผสมยา



• **หน้าแพ้เกม (Game Over):** หน้าจอแสดงผลเมื่อมอนสเตอร์บุกโจมตีหอคอยจนพลังชีวิตเหลือศูนย์ จะแสดงข้อความแสดงความพ่ายแพ้ พร้อมปุ่มตัวเลือกให้ผู้เล่นกด "ลองใหม่อีกครั้ง" เพื่อเริ่มเกมแก้ตัวในรอบใหม่ หรือกด "กลับหน้าหลัก" เพื่อออกจากด่าน



• **หน้าชนะเกม (Victory):** หน้าจอแสดงผลเมื่อผู้เล่นสามารถปกป้องหอคอยและสกัดกั้นฝูงมอนสเตอร์ได้สำเร็จจนจบภารกิจ จะแสดงข้อความแสดงความยินดี พร้อมปุ่มตัวเลือกให้ "เล่นอีกครั้ง" หรือกด "กลับหน้าหลัก"

ประโยชน์

1. ฝึกทักษะการแยกประสาท และการทำงานหลายอย่างพร้อมกัน (Multitasking)
2. พัฒนาทักษะการคิดวิเคราะห์ และการวางแผน (Strategic Thinking)
3. ฝึกความจำ และความรวดเร็วในการตอบสนอง (Memory & Reflexes)
4. ให้ความบันเทิง และคลายเครียด

ตารางแผนการทำงาน

ลำดับ	รายการ	ก.ย. W1	ก.ย. W2	ก.ย. W3	ก.ย. W4	ต.ค. W1	ต.ค. W2	ต.ค. W3	ต.ค. W4
1	ศึกษาข้อมูล และออกแบบโครงสร้างเกม								
2	ออกแบบกราฟิก , UI และหาภาพตัวละคร								
3	ลงมือเขียนโปรแกรม และพัฒนาระบบเกม								
4	ตรวจสอบการทำงาน และแก้ไขข้อผิดพลาด								
5	จัดทำเอกสารรายงาน								

บทที่ 2 ส่วนการพัฒนา

เนื้อเรื่องย่อ

ในเกม POTION PANIC (อลวนคนปรุงยา) คุณจะได้รับบทเป็น **"ศิษย์ฝึกหัดนักปรุงยา"** ที่เพิ่งเข้ามารับเวรกะดึกเฝ้าหอคอยเพียงลำพัง แต่จู่ๆ คืนที่ควรจะเงียบสงบก็พังทลายลง เมื่อรอยแยกมิติเปิดออกและฝูงมอนสเตอร์กลายพันธุ์กำลังแห่กันมาที่หอคอย! ซ้ำร้าย **"สายธารวัตถุดิบเวทมนตร์"** ที่เชื่อมต่อกับใต้ดินดันทำงานรวน มันสูบเอาสมุนไพรและแร่ธาตุต่างๆ ไหลทะลักขึ้นมาอย่างบ้าคลั่งและไม่ขาดสาย คุณไม่มีกองทหาร ไม่มีดาบวิเศษ มีเพียงหม้อต้มยาเก่าๆ กับความจำในสมอง คุณต้องตั้งสติรับมือกับความอลวนนี้ คว้าวัตถุดิบที่ไหลมาตามสายพาน โยนลงหม้อให้ถูกสูตร แล้วสาดขวดยาเวทมนตร์สกัดกั้นพวกมอนสเตอร์ให้ทันเวลา ก่อนที่หอคอยแห่งนี้จะแตก!

วิธีการเล่น

- กดปุ่มตัวเลข **1, 2, 3, 4, 5** เพื่อดึงวัตถุดิบจากสายพานในแต่ละช่องลงหม้อต้ม
- กดปุ่ม **Spacebar** เพื่อทำการผสมยา และใช้เมาส์คลิกซ้าย เพื่อเลือกขวดยาพร้อมกับคลิกเป้าหมายเพื่อปาโจมตีมอนสเตอร์

นอกจากนี้ยังมีปุ่มลัดเสริมเพื่อความคล่องตัวในการเล่น ได้แก่

- ปุ่ม **[F]** สำหรับสุ่มวัตถุดิบใหม่ทั้งสายพาน (Reroll)
- ปุ่ม **[Q], [W], [E], [R]** สำหรับสลับเลือกขวดยาบนแท่นวางยา
- ปุ่ม **[Backspace]** สำหรับนำวัตถุดิบชิ้นล่าสุด ออกจากหม้อ
- ปุ่ม **[C]** สำหรับล้างหม้อต้มทั้งหมด
- ปุ่ม **[Tab]** หรือ **[H]** เพื่อเปิดดูสมุดสูตรยา
- ปุ่ม **[M]** สำหรับสลับเปิด/ปิดเสียงเพลง
- ปุ่ม **[F11]** เพื่อสลับโหมดเต็มหน้าจอ

Storyboard

ตัวละครและไอเทม

1. มอนสเตอร์และบอสใหญ่ในสนามรบ (5 ชนิด)

ประกอบด้วยมอนสเตอร์ 5 ชนิดที่บุกโจมตีหอคอยเวทมนตร์ โดยมีจุดอ่อนตามธาตุและบอสใหญ่สลับเกราะ:

				
อสูรพฤกษา (แพ้ไฟ)	**โกเลมไฟ** (แพ้น้ำแข็ง)	**สัตว์ร้ายเกราะ** (แพ้กรด)	**อิมป์ว่องไว** (แพ้สายฟ้า)	**ราชันย์อสูรเวท** (บอสใหญ่)

2. อุปกรณ์หอคอยเวทมนตร์ & วัตถุดิบปรุงยา 4 ชนิด

อุปกรณ์หลักของนักปรุงยาในการป้องกันหอคอย และสมุนไพร/แร่ธาตุ 4 ธาตุที่ไหลมาตามสายพาน:

					
หอคอยเวทมนตร์ (Tower)	**หม้อต้มรูนโบราณ** (Cauldron)	**สมุนไพรเพลิง** (Fire)	**ผลึกน้ำแข็ง** (Ice)	**เห็ดพิษ** (Poison)	**รากสายฟ้า** (Lightning)

3. ขวดยาเวทมนตร์และสูตรผสม (7 ชนิด)

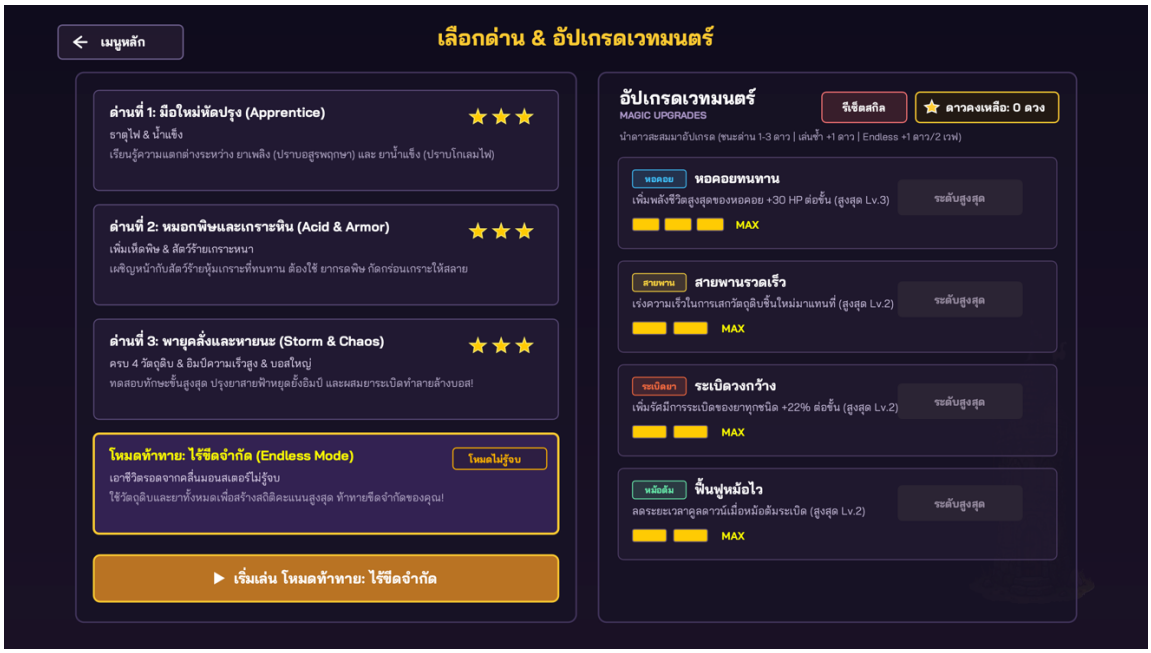
ขวดยาเวทมนตร์ธาตุต่างๆ สำหรับใช้ขว้างสกัดกั้นศัตรู รวมถึงยาระเบิดมหาเวทระดับ RARE:

						
ยาเพลิงกัลป์ (Fire Potion)	**ยาน้ำแข็ง** (Ice Potion)	**ยากรดพิษ** (Poison Potion)	**ยาสายฟ้า** (Lightning Potion)	**ยาหมอกพายุพิษ** (Frost-Venom)	**ยาระเบิดมหาเวท** (RARE! Miracle)	**ยาปรุงผิดพลาด** (Failed Smoke)

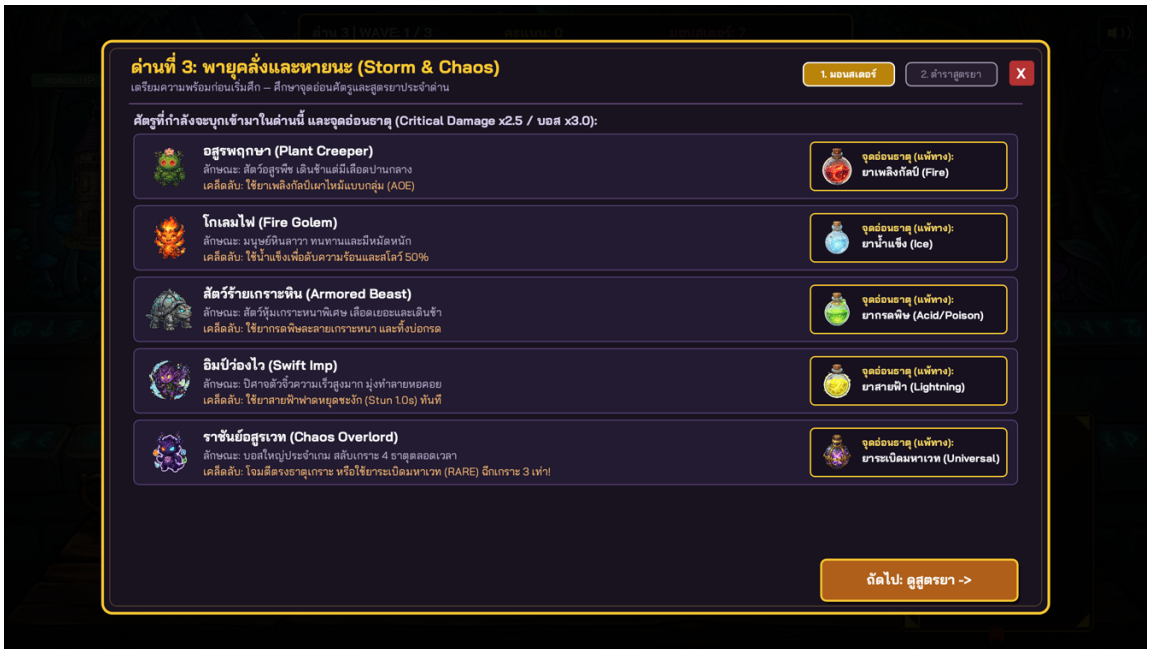
ฉากภายในเกม



• **หน้าเริ่มเกม (Main Menu):** หน้าจอเริ่มต้นของเกมที่แสดงป้ายเมนูหลัก ได้แก่ ปุ่ม "เริ่มเกม" สำหรับเข้าสู่เกม และปุ่ม "ตำราวิธีเล่น & สารานุกรม" สำหรับดูคู่มือปุ่มลัดและสูตรยาเบื้องต้น

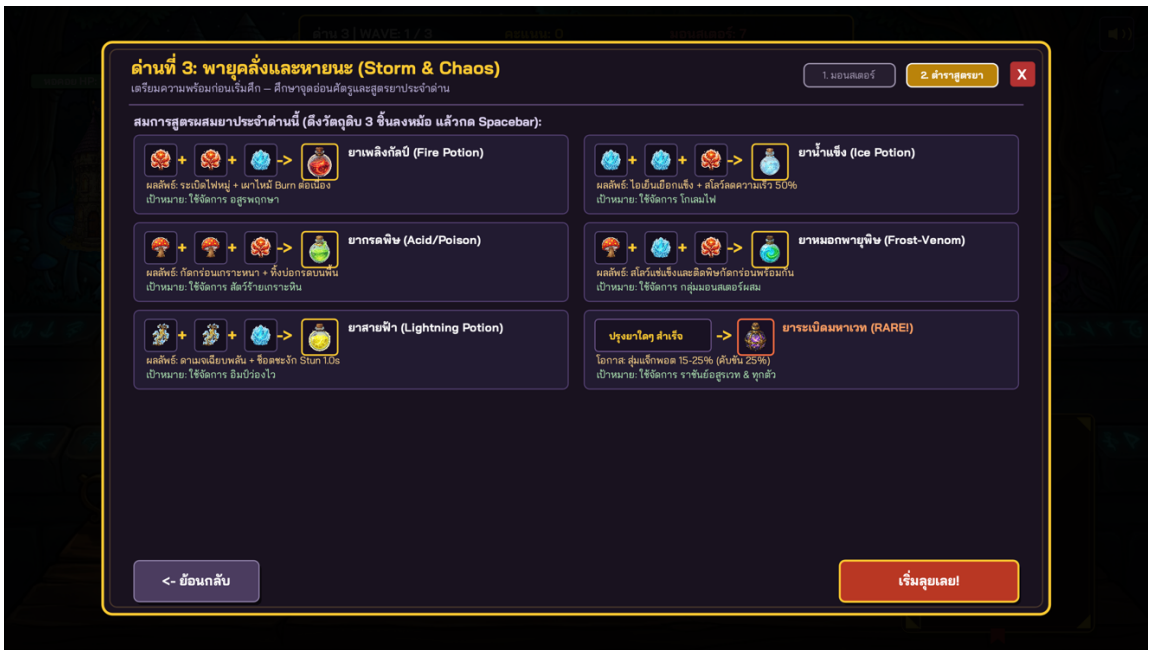


• **หน้าเลือกด่านและร้านค้า (Stage Select & Shop):** หน้าจอแสดงด่านทั้ง 4 ด่าน พร้อมร้านค้าอัปเกรดพลังชีวิตหอคอย ความเร็วสายพาน รัศมีระเบิด และปุ่มรีเซ็ตสกิลคืนดาว 100%



• **หน้าต่างสรุปข้อมูลและสอนสูตรก่อนเริ่มด่าน (Stage Briefing Overlay):**

หน้าจอแจ้งเตือนกลยุทธ์ที่จะปรากฏขึ้นอัตโนมัติก่อนเริ่มเวฟแรกของแต่ละด่าน เพื่อเตรียมความพร้อมให้แก่ผู้เล่น โดยแบ่งออกเป็น 2 หน้า ได้แก่ **หน้าที่ 1 (ข้อมูลมอนสเตอร์):** แสดงรายชื่อ จุดเด่น ค่าสถานะ และจุดอ่อนธาตุของมอนสเตอร์ทั้งหมดที่จะบุกเข้ามาในด่านนั้น เพื่อให้ผู้เล่นทราบว่าต้องใช้ยาชนิดใดโจมตีให้เกิด Critical Damage



หน้าที่ 2 (ตำราสูตรยา): แสดงสมการภาพของสูตรยาที่จำเป็นต้องใช้ในด่าน

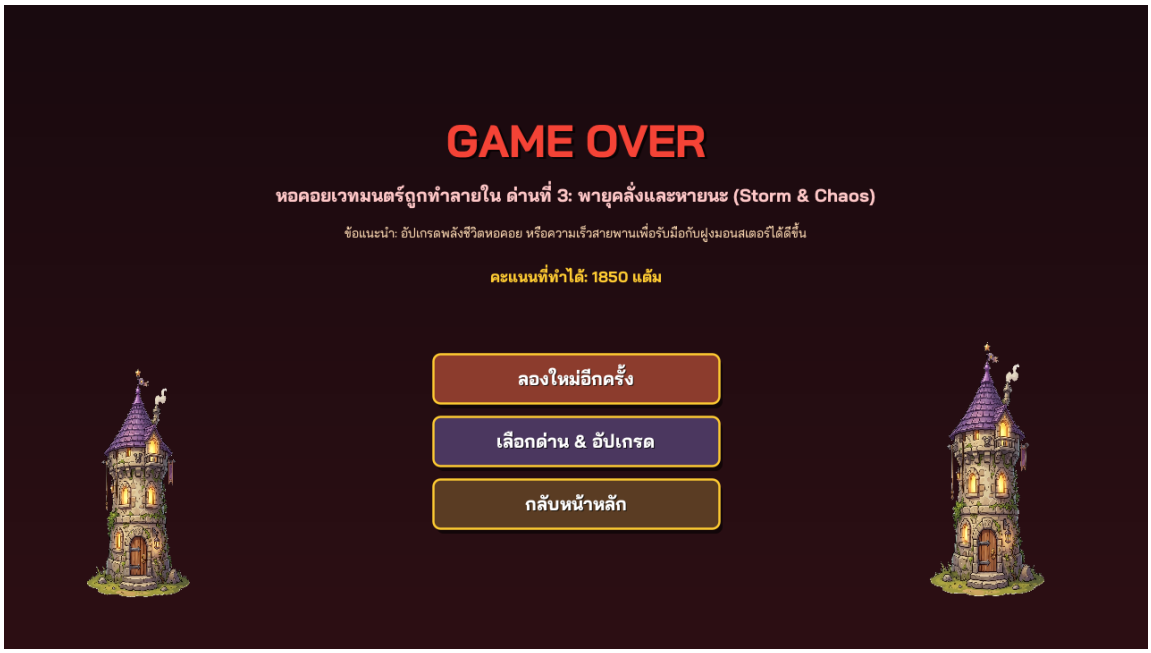


• **หน้าระหว่างเกม (Gameplay):** หน้าจอหลักขณะเล่นที่แบ่งสัดส่วนการควบคุมและการแสดงผลออกเป็น 3 โซนหลักอย่างชัดเจน:

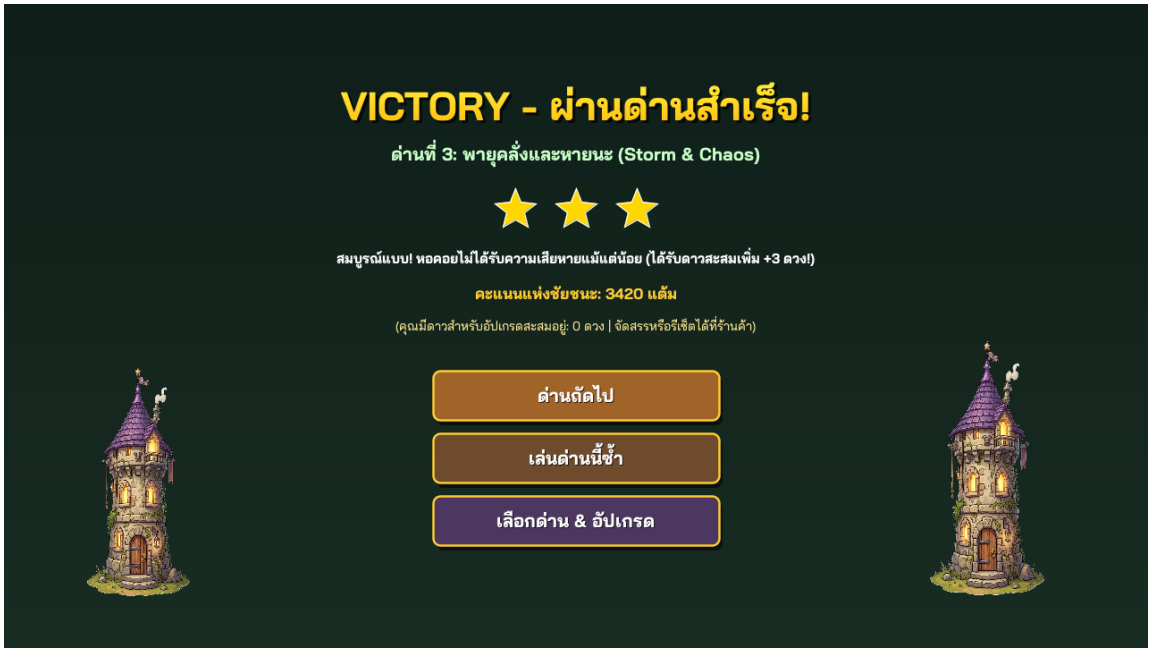
โซนบน คือพื้นที่สนามรบและหอคอยเวทมนตร์ แสดงหลอดพลังชีวิต (HP) ของหอคอยที่ผู้เล่นต้องปกป้อง เส้นทางเดินของมอนสเตอร์พร้อมป้ายบอกจุดอ่อนธาตุของแต่ละตัวเพื่อให้ผู้เล่นเลือกใช้ยาแก้ทางได้ถูกต้อง พร้อมแถบสถานะด้านบนที่แสดงคะแนน เวฟปัจจุบัน จำนวนศัตรูที่เหลือ และอัตราตัวคูณคอมโบ

โซนกลาง คือสายพานลำเลียงวัตถุดิบ 5 ช่อง ซึ่งผู้เล่นสามารถกดปุ่มตัวเลข 1–5 บนคีย์บอร์ดเพื่อดึงวัตถุดิบลงหม้อได้อย่างรวดเร็ว พร้อมปุ่มลัด F สำหรับกดสุ่มวัตถุดิบใหม่ทั้งแถวเมื่อติดคูลดาวน์

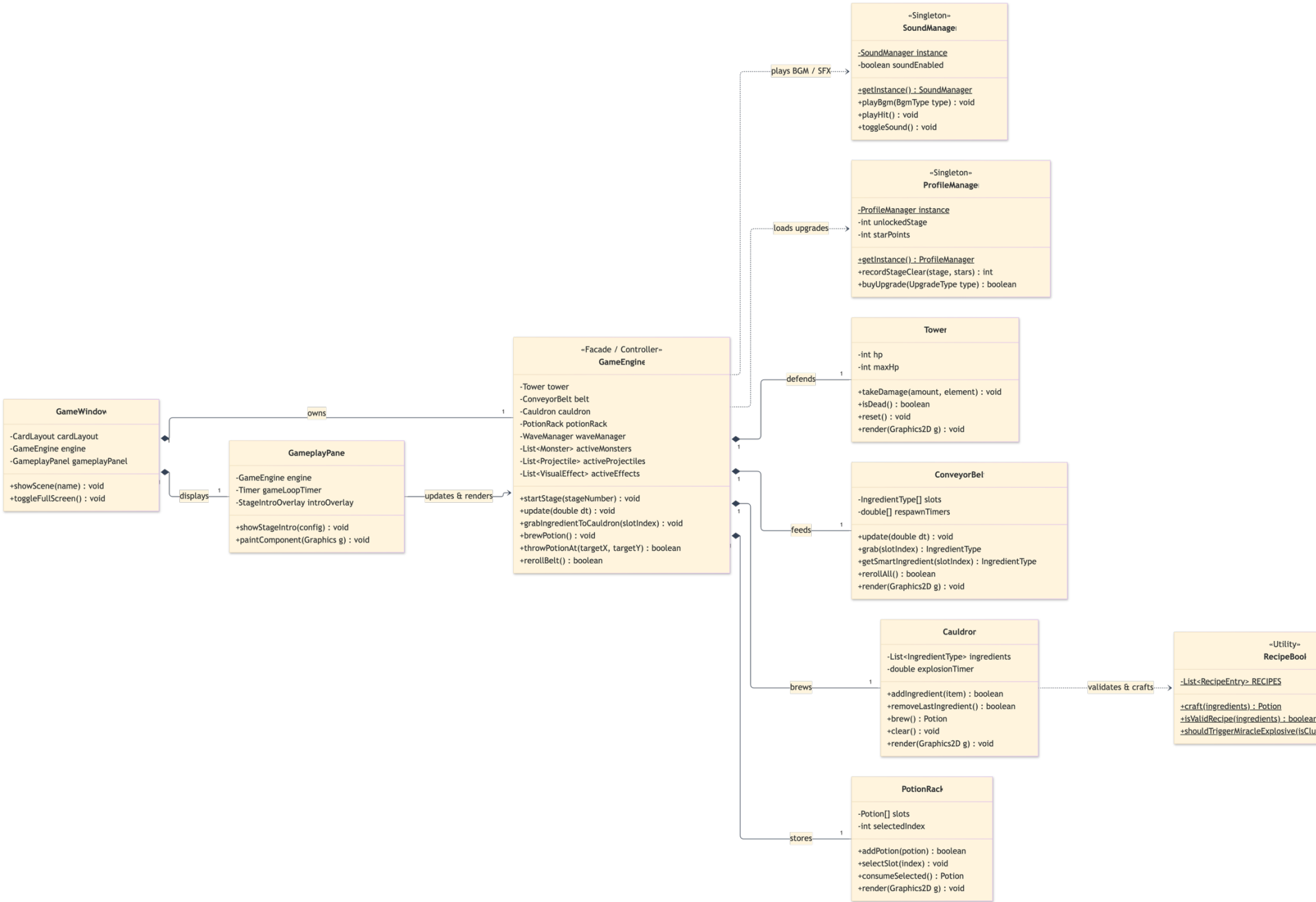
โซนล่าง คือโต๊ะทำงานของนักปรุงยา ประกอบด้วยหม้อต้มสำหรับผสมวัตถุดิบ 3 ช่อง พร้อมระบบฟองคาถาบอกใบ้สูตรยาที่จะได้ แท่นวางขวดยาที่ปรุงเสร็จแล้วจำนวน 4 ช่องที่เลือกใช้ได้ด้วยปุ่ม Q, W, E, R และสมุดตำราสูตรยาสำหรับคลิกเปิดดูวิธีผสมยาได้ตลอดเวลา



• **หน้าแพ้เกม** (Game Over): หน้าจอแสดงผลเมื่อมอนสเตอร์บุกโจมตีหอคอยจนพลังชีวิตเหลือศูนย์ จะแสดงข้อความแสดงความพ่ายแพ้ พร้อมปุ่มตัวเลือกให้ผู้เล่นกด "ลองใหม่อีกครั้ง" เพื่อเริ่มเกมแก้ตัวในรอบใหม่ หรือกด "กลับหน้าหลัก" เพื่อออกจากด่าน



• **หน้าชนะเกม** (Victory): หน้าจอแสดงผลเมื่อผู้เล่นสามารถปกป้องหอคอย และสกัดกั้นฝูงมอนเตอร์ได้สำเร็จจนจบภารกิจ จะแสดงข้อความแสดงความยินดี คำนวณดวงดาวที่ได้รับ พร้อมปุ่มตัวเลือกให้ "เล่นอีกครั้ง" หรือกด "กลับหน้าหลัก"


«Singleton»
SoundManager
-SoundManager instance
-boolean soundEnabled
+getInstance() : SoundManager
+playBgm(BgmType type) : void
+playHit() : void
+toggleSound() : void
plays BGM / SFX
«Singleton»
ProfileManager
-ProfileManager instance
-int unlockedStage
-int starPoints
+getInstance() : ProfileManager
+recordStageClear(stage, stars) : int
+buyUpgrade(UpgradeType type) : boolean
loads upgrades
«Facade / Controller»
GameEngine
-Tower tower
-ConveyorBelt belt
-Cauldron cauldron
-PotionRack potionRack
-WaveManager waveManager
-List<Monster> activeMonsters
-List<Projectile> activeProjectiles
-List<VisualEffect> activeEffects
+startStage(stageNumber) : void
+update(double dt) : void
+grabIngredientToCauldron(slotIndex) : void
+brewPotion() : void
+throwPotionAt(targetX, targetY) : boolean
+rerollBelt() : boolean
Tower
-int hp
-int maxHp
+takeDamage(amount, element) : void
+isDead() : boolean
+reset() : void
+render(Graphics2D g) : void
defends
1
1
GameWindow
-CardLayout cardLayout
-GameEngine engine
-GameplayPanel gameplayPanel
+showScene(name) : void
+toggleFullScreen() : void
owns
1
displays
1
GameplayPane
-GameEngine engine
-Timer gameLoopTimer
-StageIntroOverlay introOverlay
+showStageIntro(config) : void
+paintComponent(Graphics g) : void
updates & renders
ConveyorBelt
-IngredientType[] slots
-double[] respawnTimers
+update(double dt) : void
+grab(slotIndex) : IngredientType
+getSmartIngredient(slotIndex) : IngredientType
+rerollAll() : boolean
+render(Graphics2D g) : void
feeds
1
Cauldron
-List<IngredientType> ingredients
-double explosionTimer
+addIngredient(item) : boolean
+removeLastIngredient() : boolean
+brew() : Potion
+clear() : void
+render(Graphics2D g) : void
brews
1
validates & crafts
«Utility»
RecipeBook
-List<RecipeEntry> RECIPES
+craft(ingredients) : Potion
+isValidRecipe(ingredients) : boolean
+shouldTriggerMiracleExplosive(isClu
PotionRack
-Potion[] slots
-int selectedIndex
+addPotion(potion) : boolean
+selectSlot(index) : void
+consumeSelected() : Potion
+render(Graphics2D g) : void
stores
1

Class Diagram

แผนภาพที่ 1: สถาปัตยกรรมระบบหลัก หน้าจอ GUI และความสัมพันธ์แบบประกอบ

แผนภาพนี้แสดงโครงสร้างการควบคุมหลักของโปรแกรม ตั้งแต่ชั้นส่วนติดต่อผู้ใช้ (GUI) ไปจนถึงเอนจินเกมและอุปกรณ์บนโต๊ะปรุงยา โดยประยุกต์ใช้หลักการ Composition (ความสัมพันธ์แบบเป็นส่วนประกอบ Has-A), Encapsulation, และ Singleton Pattern ดังนี้:

1. ชั้นหน้าจอแสดงผล (GameWindow และ GameplayPanel)

• GameWindow: สืบทอดจากคลาส JFrame ทำหน้าที่เป็นหน้าต่างหลักของโปรแกรม เป็นผู้สร้างและครอบครอง (Composition) ออบเจกต์ GameEngine และ GameplayPanel พร้อมควบคุมการสลับฉากด้วย CardLayout ผ่านเมธอด showScene() และควบคุมโหมดเต็มจอด้วย toggleFullScreen()

• GameplayPanel: สืบทอดจากคลาส JPanel ทำหน้าที่ขับเคลื่อนลูปการวาดภาพกราฟิก 60 FPS ผ่านเมธอด paintComponent() และส่งต่อคำสั่งการกดแป้นพิมพ์และเมาส์จากผู้เล่นไปยัง GameEngine

2. คลาสศูนย์กลาง (GameEngine - Facade / Controller)

ทำหน้าที่ประสานงานลอจิกทั้งหมดของเกมในแต่ละเฟรมผ่านเมธอด update(dt) โดยประกอบขึ้นจากออบเจกต์ส่วนประกอบหลัก 4 ชิ้น (Composition อัตราส่วน 1:1) ได้แก่:

• Tower (หอคอยเวทมนตร์): เก็บค่าพลังชีวิต (hp, maxHp) ไว้เป็นความลับแบบ private เพื่อปกป้องข้อมูล และเปิดให้ภายนอกเรียกใช้งานผ่านเมธอด takeDamage(), isDead(), และ reset()

• ConveyorBelt (สายพานลำเลียงวัตถุดิบ): จัดการวัตถุดิบ 5 ช่อง (slots) มีอัลกอริทึมสุ่มวัตถุดิบอัจฉริยะตามจุดอ่อนของศัตรู getSmartIngredient(), การหยิบวัตถุดิบ grab(), และการสุ่มใหม่ทั้งสายพาน rerollAll()

• Cauldron (หม้อต้มรูนโบราณ): เก็บรายการวัตถุดิบ 3 ชิ้น (ingredients) ควบคุมเวลาระเบิดเมื่อผสมผิดสูตร (explosionTimer) รองรับการใส่ส่วนผสม addIngredient(), การนำชิ้นล่าสุดออก removeLastIngredient(), และการสั่งต้มยา brew() โดยเรียกใช้คลาส RecipeBook เพื่อตรวจสอบความถูกต้องของสูตร

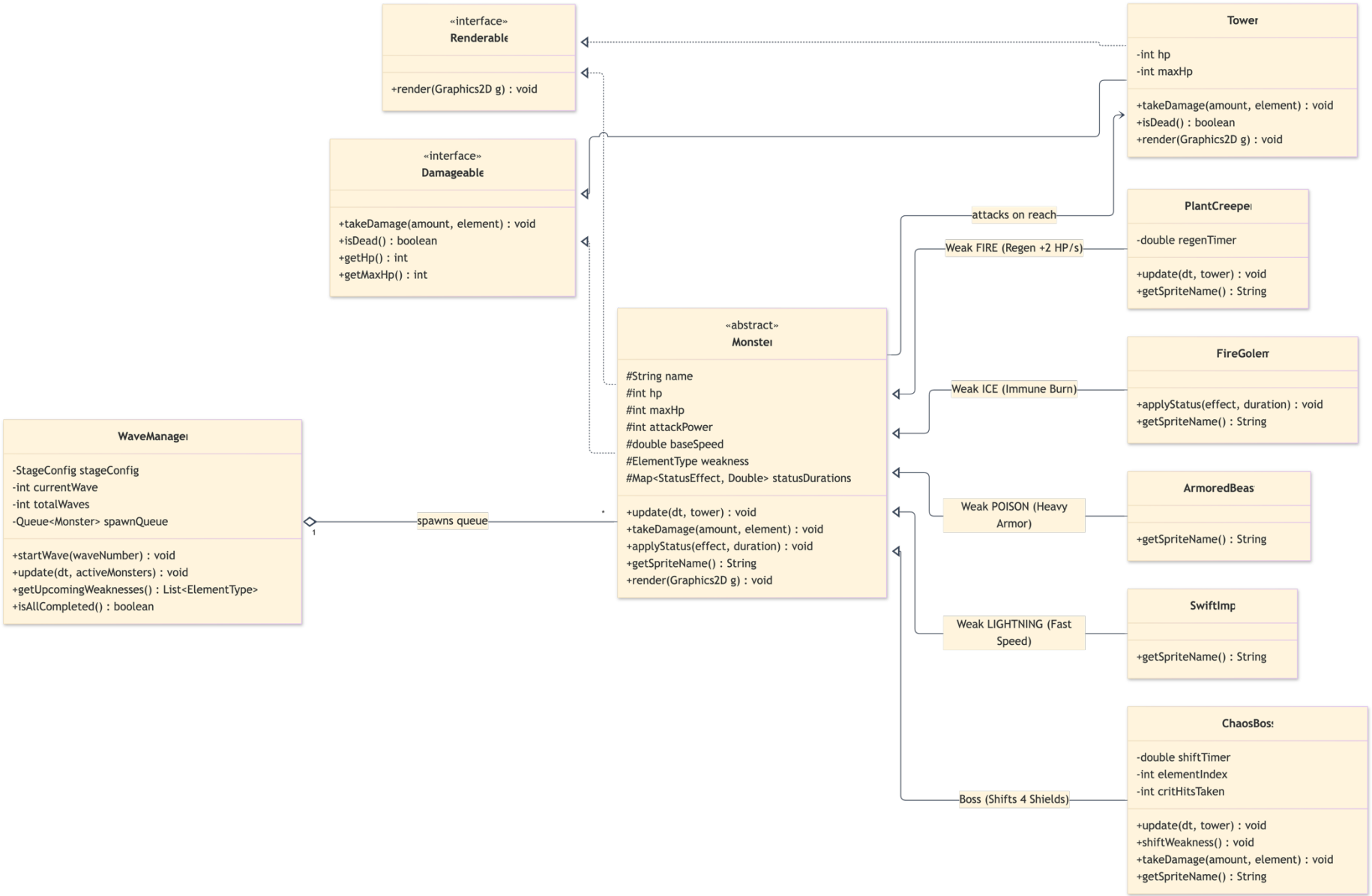
• PotionRack (ชั้นวางขวดยาพร้อมใช้): จัดการช่องเก็บยา 4 ช่อง (slots) รองรับการรับยาเข้าชั้นวาง addPotion(), การเลือกสล็อตด้วยปุ่มลัด Q/W/E/R ผ่าน selectSlot(), และการนำยาออกไปใช้งาน consumeSelected()

3. คลาสจัดการระดับโลกแบบ Singleton (SoundManager และ ProfileManager)

ทั้งสองคลาสใช้รูปแบบ Singleton Design Pattern โดยซ่อน Constructor เป็น private และเปิดให้เข้าถึงผ่านเมธอด static ชื่อ getInstance() เพื่อให้มีอินสแตนซ์เพียงหนึ่งเดียวตลอดการทำงาน:

• SoundManager: ควบคุมระบบสังเคราะห์เสียงดนตรีประกอบ (BGM) และเสียงเอฟเฟกต์เวทมนตร์ (SFX) ทั้งหมดในเกม

• ProfileManager: จัดการบันทึกจำนวนดาวที่ได้รับ บันทึกการปลดล็อกด่าน และบันทึกระดับการอัปเกรดความสามารถทั้ง 4 สาย


«interface»
Renderable
+render(Graphics2D g) : void
«interface»
Damageable
+takeDamage(amount, element) : void
+isDead() : boolean
+getHp() : int
+getMaxHp() : int
Tower
-int hp
-int maxHp
+takeDamage(amount, element) : void
+isDead() : boolean
+render(Graphics2D g) : void
attacks on reach
PlantCreeper
-double regenTimer
+update(dt, tower) : void
+getSpriteName() : String
Weak FIRE (Regen +2 HP/s)
«abstract»
Monster
#String name
#int hp
#int maxHp
#int attackPower
#double baseSpeed
#ElementType weakness
#Map<StatusEffect, Double> statusDurations
+update(dt, tower) : void
+takeDamage(amount, element) : void
+applyStatus(effect, duration) : void
+getSpriteName() : String
+render(Graphics2D g) : void
FireGolem
+applyStatus(effect, duration) : void
+getSpriteName() : String
Weak ICE (Immune Burn)
ArmoredBeas
+getSpriteName() : String
Weak POISON (Heavy Armor)
SwiftImp
+getSpriteName() : String
Weak LIGHTNING (Fast Speed)
ChaosBos
-double shiftTimer
-int elementIndex
-int critHitsTaken
+update(dt, tower) : void
+shiftWeakness() : void
+takeDamage(amount, element) : void
+getSpriteName() : String
Boss (Shifts 4 Shields)
WaveManage
-StageConfig stageConfig
-int currentWave
-int totalWaves
-Queue<Monster> spawnQueue
+startWave(waveNumber) : void
+update(dt, activeMonsters) : void
+getUpcomingWeaknesses() : List<ElementType>
+isAllCompleted() : boolean
spawns queue
1
*

แผนภาพที่ 2: ระบบปล่อยเวฟ อินเทอร์เฟซ และสายการสืบทอดมอนสเตอร์

แผนภาพส่วนที่ 2 แสดงการประยุกต์ใช้หลักการ Abstraction (อินเทอร์เฟซและคลาสนามธรรม) และ Inheritance (การสืบทอดคุณสมบัติ) ในระบบการต่อสู้และฝูงศัตรู ดังนี้:

1. อินเทอร์เฟซมาตรฐาน (Damageable และ Renderable)

• Damageable (อินเทอร์เฟซรับความเสียหาย): กำหนดข้อตกลงมาตรฐานสำหรับออบเจกต์ที่ถูกทำลายได้ ได้แก่ เมธอด takeDamage(), isDead(), getHp(), และ getMaxHp() โดยทั้งคลาส Tower และ Monster นำไปพัฒนาต่อ (Implement) ทำให้ระบบสามารถจัดการออบเจกต์ที่มีหลอดเลือดได้ด้วยมาตรฐานเดียวกัน

• Renderable (อินเทอร์เฟซการวาดภาพ): กำหนดเมธอด render(Graphics2D g) สำหรับออบเจกต์ทุกชนิดที่ต้องแสดงผลกราฟิกลงบนหน้าจอ

2. คลาสจัดการรอบการบุก (WaveManager)

ทำหน้าที่ควบคุมลำดับการปล่อยมอนสเตอร์ในแต่ละเวฟ โดยถือครองคิวมอนสเตอร์ (spawnQueue) ในรูปแบบ Aggregation และส่งข้อมูลจุดอ่อนของมอนสเตอร์ที่กำลังจะมา (getUpcomingWeaknesses()) ให้สายพานเตรียมวัตถุดิบแก้ทางล่วงหน้า

3. คลาสแม่นามธรรม (Monster - Abstract Class)

รวบรวมคุณสมบัติพื้นฐานของมอนสเตอร์ทุกตัวไว้ในตัวแปรแบบ protected ได้แก่ ชื่อ (name), พลังชีวิต (hp, maxHp), พลังโจมตี (attackPower), ความเร็วพื้นฐาน (baseSpeed), และจุดอ่อนธาตุ (weakness) พร้อมเมธอดการเดินเข้าตีหอคอย update(), การคำนวณดาเมจแพ้ทางธาตุ 2.5 เท่า takeDamage(), และเมธอดนามธรรม getSpriteName()

4. คลาสลูกมอนสเตอร์ทั้ง 5 ชนิด (Inheritance & Method Overriding)

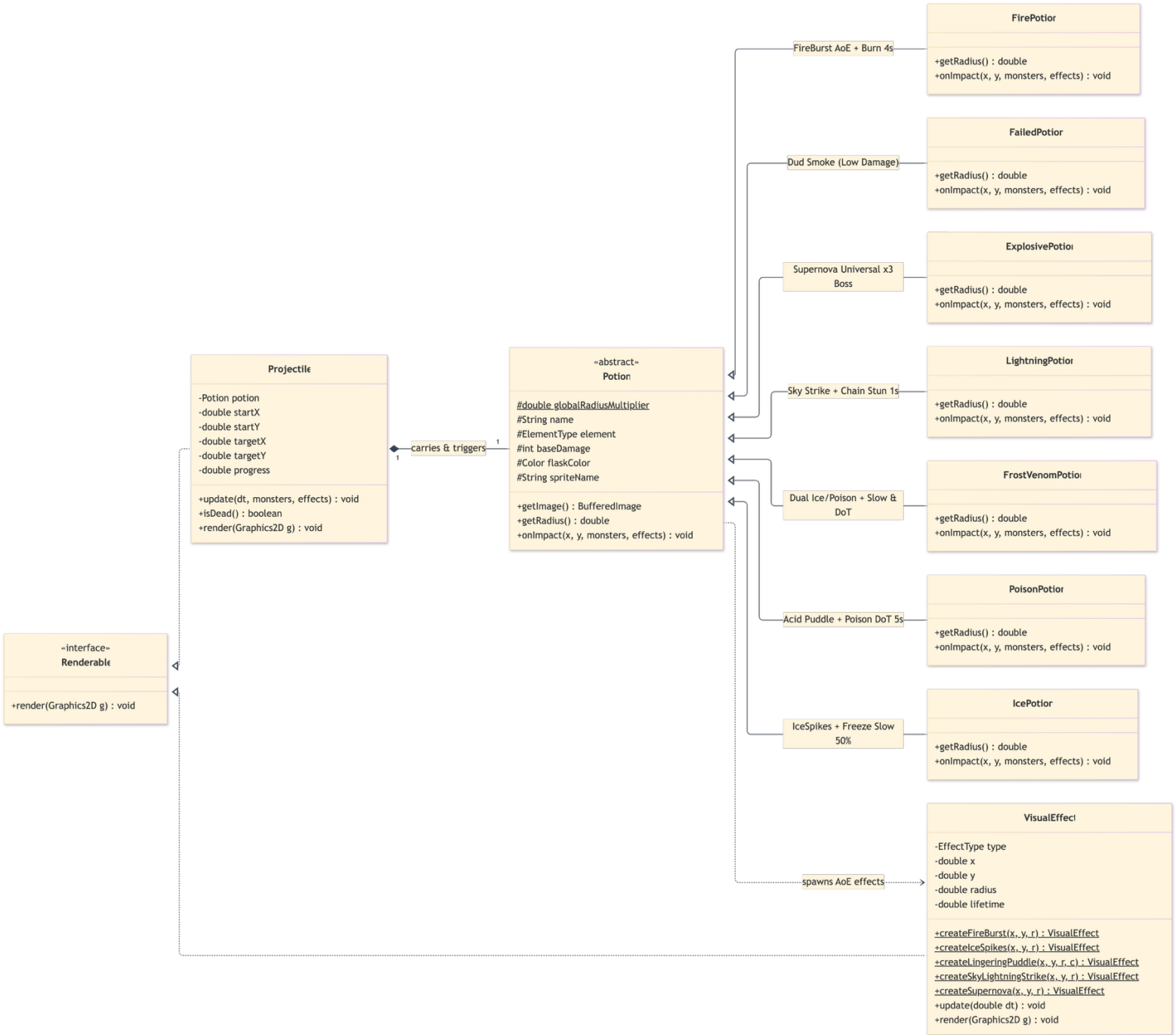
• PlantCreeper (อสูรพฤกษา): แพ้ธาตุไฟ (FIRE) ทำการ Override เมธอด update() เพื่อฟื้นฟูพลังชีวิตตนเอง +2 HP ต่อวินาทีขณะเดิน

• FireGolem (โกเลมเพลิง): แพ้ธาตุน้ำแข็ง (ICE) ทำการ Override เมธอด applyStatus() เพื่อสร้างภูมิคุ้มกันไม่ติดสถานะไฟไหม้ (BURN)

• ArmoredBeast (สัตว์ร้ายหุ้มเกราะ): เกราะหนา พลังชีวิตสูง แพ้ยากรดพิษ (POISON) ที่ช่วยละลายเกราะ

• SwiftImp (ภูตพายุว่องไว): เคลื่อนที่รวดเร็ว แพ้ยาสายฟ้า (LIGHTNING) ที่ทำให้ชะงัก (Stun)

• ChaosBoss (ราชันย์อสูรเวท - บอสใหญ่): บอสประจำด่านสุดท้าย ทำการ Override ทั้ง update() และ takeDamage() พร้อมเมธอดพิเศษ shiftWeakness() สำหรับสลับเกราะจุดอ่อนหมุนเวียนครบ 4 ธาตุตลอดการต่อสู้


«interface»
Renderable
+render(Graphics2D g) : void
Projectile
-Potion potion
-double startX
-double startY
-double targetX
-double targetY
-double progress
+update(dt, monsters, effects) : void
+isDead() : boolean
+render(Graphics2D g) : void
carries & triggers
1
1
«abstract»
Potion
#double globalRadiusMultiplier
#String name
#ElementType element
#int baseDamage
#Color flaskColor
#String spriteName
+getImage() : BufferedImage
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
FireBurst AoE + Burn 4s
FirePotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
Dud Smoke (Low Damage)
FailedPotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
Supernova Universal x3 Boss
ExplosivePotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
Sky Strike + Chain Stun 1s
LightningPotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
Dual Ice/Poison + Slow & DoT
FrostVenomPotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
Acid Puddle + Poison DoT 5s
PoisonPotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
IceSpikes + Freeze Slow 50%
IcePotion
+getRadius() : double
+onImpact(x, y, monsters, effects) : void
spawns AoE effects
VisualEffect
-EffectType type
-double x
-double y
-double radius
-double lifetime
+createFireBurst(x, y, r) : VisualEffect
+createIceSpikes(x, y, r) : VisualEffect
+createLingeringPuddle(x, y, r, c) : VisualEffect
+createSkyLightningStrike(x, y, r) : VisualEffect
+createSupernova(x, y, r) : VisualEffect
+update(double dt) : void
+render(Graphics2D g) : void

แผนภาพที่ 3: วิถีขวดยา เอฟเฟกต์สนามรบ และสายพหุสัณฐานขวดยาเวทมนตร์

แผนภาพส่วนที่ 3 แสดงการประยุกต์ใช้หลักการ Polymorphism (พหุสัณฐาน) และ Dynamic Method Dispatch ในระบบขวดยาและการระเบิด ดังนี้:

1. คลาสวิถีขวดยาและเอฟเฟกต์ภาพ (Projectile และ VisualEffect)

• Projectile: สืบทอดอินเทอร์เฟซ Renderable ทำหน้าที่คำนวณวิถีโค้งพาราโบลาของขวดยาที่ลอยกลางอากาศ และเมื่อเดินทางถึงพิกัดเป้าหมาย จะเรียกเมธอด onImpact() ของขวดยาใบนั้นทันที

• VisualEffect: คลาสจัดการเอฟเฟกต์ภาพสนามรบ เช่น เสาเพลิง (FireBurst), หนามน้ำแข็ง (IceSpikes), บ่อกรดพิษ (LingeringPuddle), สายฟ้าฟาด (SkyLightningStrike), และระเบิดซูเปอร์โนวา (Supernova)

2. คลาสแม่นามธรรม (Potion - Abstract Class)

กำหนดโครงสร้างร่วมของขวดยาทุกชนิด ได้แก่ ชื่อยา (name), ธาตุประจำยา (element), ดาเมจพื้นฐาน (baseDamage), และสีน้ำยา (flaskColor) พร้อมกำหนดเมธอดนามธรรม getRadius() และ onImpact(x, y, monsters, effects) เพื่อบังคับให้ขวดยาแต่ละชนิดไปกำหนดพฤติกรรมการระเบิดของตนเอง

3. คลาสลูกขวดยาเวทมนตร์ทั้ง 7 ชนิด (Polymorphism & Method Overriding)

คลาสลูกทุกตัวทำการ Override เมธอด getRadius() และ onImpact() ทำให้เมื่อสั่งขว้างยา ระบบจะแสดงผลลัพธ์ตามชนิดขวดยาโดยอัตโนมัติ:

• FirePotion (ยาเพลิงกัลป์): ระเบิดเสาเพลิงวงกว้าง และติดสถานะเผาไหม้ (Burn) ต่อเนื่อง 4 วินาที

• IcePotion (ยาน้ำแข็ง): ระเบิดหนามน้ำแข็ง และติดสถานะแช่แข็ง (Freeze) ลดความเร็วลง 50% นาน 4 วินาที

• PoisonPotion (ยากรดพิษ): สร้างบ่อกรดพิษค้างบนพื้น กัดกร่อนเกราะและลดเลือดต่อเนื่อง (Poison DoT) 5 วินาที

• LightningPotion (ยาสายฟ้า): สายฟ้าผ่าจากฟ้า ดาเมจสูงพร้อมช็อตให้หยุดนิ่ง (Stun) 1 วินาที

• FrostVenomPotion (ยาหมอกพายุพิษ): ยาผสาน 2 ธาตุ ส่งผลทั้งสโลว์และติดพิษในขวดเดียว

• ExplosivePotion (ยาระเบิดมหาเวท - Miracle RARE): ระเบิดมหาเวทธาตุสากล รัศมีกว้างที่สุด เจาะเกราะทุกธาตุ และทำดาเมจบอสใหญ่ 3 เท่า

• FailedPotion (ยาปรุงผิดพลาด): ยาปรุงผิดพลาดที่เกิดจากส่วนผสมไม่ตรงตำรา ดาเมจต่ำและเกิดเพียงควันระเบิด

<u>รูปแบบ Application / Applet</u>

- **รูปแบบ:** Java Standalone Desktop Application (สร้างด้วย Java Swing & AWT)

- **ภาษา:** Java (JDK 21)

- **GUI:** javax.swing (JFrame, JPanel, CardLayout) และ java.awt.Graphics2D

- **ระบบเสียง:** javax.sound.sampled (SourceDataLine Synthesizer Engine)

โครงการนี้เลือกพัฒนาในรูปแบบ **Standalone Desktop Application** โดยมีฟังก์ชัน main() ในคลาส Main เป็นจุดเริ่มต้นโปรแกรม และไม่ได้ใช้เทคโนโลยี Java Applet เนื่องจาก Applet ได้ถูกยกเลิก (Deprecated และ Removed) ออกจากภาษา Java ยุคใหม่อย่างถาวรตั้งแต่ Java 11
เนื่องจากปัญหาด้านความปลอดภัยของเบราว์เซอร์ การพัฒนาเป็น Desktop Application จึงเป็นแนวทางมาตรฐานที่ถูกต้อง มีความปลอดภัยสูง เข้าถึงการเรนเดอร์ระดับฮาร์ดแวร์เร่งความเร็วได้เต็มที่ และสามารถสร้างไฟล์ Executable JAR ดับเบิลคลิกเล่นได้บนทุกระบบปฏิบัติการ

ตัวอย่างโปรแกรมบางส่วน

Constructor

Constructor ทำหน้าที่กำหนดค่าเริ่มต้นให้แก่ออบเจกต์เมื่อถูกสร้างขึ้นใหม่ ป้องกันข้อผิดพลาดจากค่าว่าง (Null) และเตรียมความพร้อมของข้อมูล

```
// ตัวอย่าง Constructor ในคลาส ChaosBoss.java
public ChaosBoss(double startX, double startY) {
    // เรียก Constructor ของคลาสแม่ (Monster) ด้วยคำสั่ง super
    super("ราชันย์อสูรเวท (Chaos Overlord)", 420, 0.28, 25, ElementType.FIRE,
startX, startY);

    this.width = 96;
    this.height = 96;
}
```

ในตัวอย่างนี้ คอนสตรัคเตอร์ของ ChaosBoss จะเรียกใช้ super() เพื่อส่งค่าชื่อ เลือด 420 ความเร็ว 0.28 พลังโจมตี 25 และจุดอ่อนเริ่มต้นที่ธาตุไฟ ไปยังคอนสตรัคเตอร์ของคลาสแม่ Monster พร้อมกำหนดขนาดกว้างยาว 96x96 พิกเซล

Encapsulation

Encapsulation คือการซ่อนข้อมูลสถานะภายในไว้เป็นความลับด้วยการกำหนดแอตทริบิวต์เป็น private และเปิดให้ภายนอกเข้าถึงผ่านเมธอด Getter และ Setter ที่มีการตรวจสอบความถูกต้อง

```
// ตัวอย่าง Encapsulation ในคลาส Tower.java
public class Tower implements Damageable, Renderable {
    private int hp;
    private int maxHp;

    public void takeDamage(int amount, ElementType element) {
        // ควบคุมไม่ให้พลังชีวิตติดลบต่ำกว่า 0
        this.hp = Math.max(0, this.hp - amount);
    }

    public int getHp() { return hp; }
    public int getMaxHp() { return maxHp; }
}
```

ตัวแปร hp และ maxHp ถูกห่อหุ้มไว้เป็น private ทำให้คลาสอื่นไม่สามารถแก้ไขเลือดได้โดยตรง แต่ต้องเรียกผ่าน takeDamage() ซึ่งมีคำสั่ง Math.max(0, ...) เพื่อป้องกันไม่ให้เลือดหอคอยติดลบ

Composition

Composition คือการสร้างคลาสหลักโดยประกอบขึ้นจากออบเจกต์ของคลาสย่อยๆ (Has-A Relationship) เพื่อแบ่งแยกหน้าที่ความรับผิดชอบอย่างเป็นระเบียบ

```java
// ตัวอย่าง Composition ในคลาส GameEngine.java
public class GameEngine {
    // GameEngine "ประกอบด้วย" หอคอย, สายพาน, หม้อต้ม, และแท่นวางยา
    private final Tower tower;
    private final ConveyorBelt belt;
    private final Cauldron cauldron;
    private final PotionRack potionRack;
    private final WaveManager waveManager;

    public GameEngine() {
        this.tower = new Tower(Constants.TOWER_MAX_HP);
        this.belt = new ConveyorBelt();
        this.cauldron = new Cauldron(Constants.ZONE_BOT_X, Constants.ZONE_BOT_Y
+ 15);
        this.potionRack = new PotionRack(Constants.ZONE_BOT_X + 220,
Constants.ZONE_BOT_Y + 15);
        this.waveManager = new WaveManager();
    }
}
```

คลาส GameEngine ทำหน้าที่เป็นผู้ควบคุมส่วนกลางโดยประกอบออบเจกต์ของหอคอย สายพาน หม้อต้ม และแท่นวางยา เข้ามาทำงานร่วมกันอย่างเป็นระบบ

Polymorphism

Polymorphism คือความสามารถของออบเจกต์ในคลาสลูกต่างๆ ในการตอบสนองต่อเมธอดชื่อเดียวกันด้วยพฤติกรรมเฉพาะตัวที่แตกต่างกันผ่านการ Override

```java
// ตัวอย่าง Polymorphism ในคลาสลูกของ Potion.java
// คลาส FirePotion: ปะทุเป็นเสาเพลิงและติดสถานะเผาไหม้ (Burn)
public class FirePotion extends Potion {
    @Override
    public void onImpact(double impactX, double impactY, List<Monster>
allMonsters, List<VisualEffect> effects) {
        effects.add(VisualEffect.createFireBurst(impactX, impactY,
getRadius()));
        // ทำดาเมจธาตุไฟและเผาไหม้ต่อเนื่อง
    }
}

// คลาส IcePotion: ปะทุเป็นหนามผลึกน้ำแข็งและติดสถานะแช่แข็ง (Freeze)
public class IcePotion extends Potion {
    @Override
    public void onImpact(double impactX, double impactY, List<Monster>
allMonsters, List<VisualEffect> effects) {
        effects.add(VisualEffect.createIceSpikes(impactX, impactY,
getRadius()));
        // ทำดาเมจธาตุน้ำแข็งและสโลว์ 50%
    }
}
```

เมื่อ GameEngine สั่ง p.onImpact() ขวดยาแต่ละชนิดจะทำงานไม่เหมือนกัน ยาเพลิงจะสร้างเสาไฟและเผาไหม้ ส่วนยาน้ำแข็งจะสร้างหนามน้ำแข็งและสโลว์มอนสเตอร์ ทั้งที่เป็นการเรียกเมธอดเดียวกัน

Abstract

Abstract ใช้สำหรับสร้างคลาสต้นแบบและอินเทอร์เฟซเพื่อกำหนดโครงสร้างร่วมและข้อตกลงที่คลาสลูก จำเป็นต้องนำไปพัฒนาต่อ

```java
// ตัวอย่าง Abstract Class ในคลาส Potion.java
public abstract class Potion implements Renderable {
    protected String name;
    protected ElementType element;
    protected int baseDamage;

    // บังคับให้คลาสลูกต้องเขียนการระเบิดตามธาตุของตัวเอง
    public abstract void onImpact(double impactX, double impactY,List<Monster>
allMonsters, List<VisualEffect> effects);
}
```

คลาส Potion ถูกกำหนดให้เป็น abstract class เพื่อป้องกันไม่ให้ถูก new เป็นออบเจกต์เปล่าๆ และบังคับให้คลาสลูกทุกตัวต้องเขียนโค้ดในเมธอด onImpact() ของตนเอง

Inheritance

Inheritance คือการสืบทอดคุณสมบัติและพฤติกรรมจากคลาสแม่มาสู่คลาสลูกด้วยคีย์เวิร์ด extends เพื่อลดความซ้ำซ้อนของโค้ด

```
// ตัวอย่าง Inheritance ในคลาส SwiftImp.java
public class SwiftImp extends Monster {
    public SwiftImp(double x, double y) {
        // สืบทอดฟังก์ชันการเดิน แอนิเมชัน และระบบหลอดเลือดจาก Monster ทั้งหมด
        super("อิมป์ว่องไว (Swift Imp)", 65, 1.15, 12, ElementType.LIGHTNING, x,
y);
    }
}
```

คลาส SwiftImp สืบทอดคุณสมบัติมาจากคลาสแม่ Monster ทำให้ได้รับระบบการเดิน ระบบอนิเมชัน และการรับดาเมจมาใช้ทันทีโดยไม่ต้องเขียนโค้ดซ้ำใหม่

<u>ส่วนประกอบ GUI</u>

โครงสร้างของหน้าจอ GUI พัฒนาขึ้นโดยใช้ลำดับชั้นคอมโพเนนต์ (Component Hierarchy) ของ Java Swing ตามมาตรฐาน Desktop Application ดังนี้:

• GameWindow (JFrame): หน้าต่างหลักของโปรแกรมขนาดเริ่มต้น 1280x720 พิกเซล ควบคุมการสลับฉากระหว่างหน้าจอต่าง ๆ ด้วย CardLayout รองรับการปรับขนาดย่อ/ขยายหน้าต่าง และระบบแสดงผลเต็มจอ (Fullscreen Mode)

• rootPanel (JPanel): คอนเทนเนอร์หลักที่บรรจุหน้าจอฉากต่าง ๆ ในรูปแบบการ์ด ได้แก่ MainMenuPanel, StageSelectPanel, GameplayPanel, VictoryPanel, และ GameOverPanel

• GameplayPanel (JPanel): แคนวาสหลักในการขับเคลื่อนเกมเพลย์ โดยทำการ Override เมธอด paintComponent(Graphics g) และใช้ java.awt.Graphics2D วาดกราฟิกแบบ Double Buffering ที่ความเร็ว 60 เฟรมต่อวินาที พร้อมระบบ ScaleHelper คำนวณอัตราส่วนการย่อ/ขยายพิกัดเสมือนโดยคงสัดส่วนภาพ 16:9 ไว้อย่างถูกต้อง

• การแบ่งพื้นที่ 3 โซนบนหน้าจอเกมเพลย์:

1. โซนบน (Battlefield - ความสูง 280 px): พื้นที่เดินทัพของมอนสเตอร์ แถบเลือดมอนสเตอร์ และหอคอยเวทมนตร์

2. โซนกลาง (Conveyor - ความสูง 160 px): แท่นวางสายพานวัตถุดิบ 5 ช่อง และปุ่มคำสั่งสุ่มสายพาน [F]

3. โซนล่าง (Alchemy Table - ความสูง 280 px): โต๊ะปรุงยา หม้อต้มรูนโบราณ แท่นวางขวดยาพร้อมใช้ 4 ช่อง และกระดานสูตรยาเร่งด่วน



ภาพการจัดวางองค์ประกอบส่วนติดต่อผู้ใช้ (GUI) ของหน้าจอ GameplayPanel แบ่งออกเป็น 3 โซนหลัก

Event handling

เกมนี้มีระบบการจัดการเหตุการณ์ (**Event Handling**) 2 รูปแบบหลัก เพื่อรองรับการควบคุมแบบ Real-time:

1. Keyboard Event Handling (KeyListener / KeyAdapter & KeyEventDispatcher):

ใช้ดักจับสัญญาณการกดปุ่มจากผู้เล่น เพื่อสั่งการลอจิกของเกมได้อย่างรวดเร็ว โดยมีการแยกประเภทการควบคุมผ่านปุ่มลัด (**Hotkeys**) ดังตาราง:

ปุ่ม (Key Binding)	การทำงานในระบบเกม	Event Listener / Method
1, 2, 3, 4, 5	หยิบวัตถุดิบจากสายพานช่องที่ 1-5 เข้าสู่หม้อต้ม	keyPressed -> grabIngredientToCauldron()
Spacebar	ต้มยา/ผสมวัตถุดิบ 3 ชิ้นในหม้อ	keyPressed -> brewPotion()
Q, W, E, R	เลือกหยิบขวดยาในช่องพักยา (Slot 1-4) เตรียมขว้าง	keyPressed -> selectSlot()
Backspace	หยิบวัตถุดิบชิ้นล่าสุดออกจากหม้อต้ม	keyPressed -> removeLastIngredient()
C	ล้างหม้อต้ม (เทวัตถุดิบทั้งหมดทิ้ง)	keyPressed -> cauldron.clear()
F	สุ่มสายพานวัตถุดิบใหม่ (Reroll Belt)	keyPressed -> rerollBelt()
Tab / H	เปิด/ปิด สมุดบันทึกสูตรปรุงยา (Recipe Book)	keyPressed -> toggleRecipeBook()
F11	สลับโหมดเต็มจอ / หน้าต่างปกติ (Toggle Fullscreen)	KeyEventDispatcher -> toggleFullscreen()
M	เปิด / ปิด เสียงประกอบและเพลง (Mute Sound)	KeyEventDispatcher -> toggleMute()
Esc	หยุดเกมชั่วคราว หรือกลับสู่หน้าเมนูหลัก	keyPressed -> showScene("MENU")

2 Mouse Event Handling (MouseListener & MouseMotionAdapter):

ใช้ mouseMoved ในการคำนวณพิกัดเมาส์เสมือน (Virtual Coordinates) เพื่อวาดวงแหวนล็อกเป้าหมาย และใช้ mousePressed เพื่อตรวจสอบการคลิกปุ่มบนหน้าจอ หรือการคลิกบนสนามรบเพื่อเล็งและขว้างขวดยาไปยังเป้าหมาย

```
// การดักจับ Event ของเมาส์พร้อมการแปลงพิกัดเสมือน (Virtual Coordinates) ใน GameplayPanel
MouseAdapter mouseAdapter = new MouseAdapter() {
    private void updateMouseCoords(MouseEvent e) {
        // แปลงพิกัดจริงของหน้าจอเป็นพิกัดเสมือน 1280x720 เพื่อความแม่นยำทุกขนาดจอ
        Point v = ScaleHelper.toVirtual(e.getX(), e.getY(), getWidth(), getHeight());
        mouseX = v.x;
        mouseY = v.y;
    }

    @Override
    public void mouseMoved(MouseEvent e) {
        updateMouseCoords(e);
        repaint(); // อัปเดตตำแหน่งวงแหวนเล็งเป้า
    }

    @Override
    public void mousePressed(MouseEvent e) {
        requestFocusInWindow();
        updateMouseCoords(e);
        engine.throwSelectedPotion(mouseX, mouseY); // ขว้างยาไปยังพิกัดเป้าหมาย
    }
};
addMouseListener(mouseAdapter);
addMouseMotionListener(mouseAdapter);
```

<u>อัลกอริทึมที่สำคัญในโปรแกรม</u>

1.อัลกอริทึมตรวจสอบสูตรผสมยา 3 วัตถุดิบ (Order-Independent Matching):

ประยุกต์ใช้โครงสร้างข้อมูลตารางแจกแจงความถี่ (**Frequency Map: Map<IngredientType, Integer>)** ในการเปรียบเทียบจำนวนวัตถุดิบ 3 ชิ้นในหม้อต้มกับสูตรยาในตำรา ช่วยให้ผู้เล่นสามารถหยิบวัตถุดิบชิ้นใดลงก่อนหรือหลังก็ได้ (เช่น สมุนไพรเพลิง 2 ชิ้น + ผลึกน้ำแข็ง 1 ชิ้น จะได้ยาเพลิงเสมอ) โดยอัลกอริทึมมีความซับซ้อนเชิงเวลาเพียง **O(**1) ซึ่งมีประสิทธิภาพสูงกว่าการตรวจสอบลำดับสับเปลี่ยน (**Permutation** 3! = 6 รูปแบบ)

2. อัลกอริทึมสุ่มวัตถุดิบอัจฉริยะตามภัยคุกคาม (Smart Threat-Aware Spawning):

ระบบสายพานจะวิเคราะห์บริบทสนามรบแบบ **Real-time** โดยคำนวณหาตำแหน่งมอนสเตอร์ที่อยู่ใกล้หอคอยที่สุด (**Min X-Coordinate)** และตรวจสอบวัตถุดิบที่ค้างอยู่ในหม้อต้ม จากนั้นใช้วิธีการกำหนดค่าน้ำหนักความน่าจะเป็นแบบพลวัต (**Dynamic Weighted Probability)** เร่งสุ่มวัตถุดิบชิ้นที่จำเป็นลงมาบนสายพานด้วยโอกาสสูงถึง 85% ช่วยลดโอกาสที่เกมจะติดขัด (**Unwinnable RNG)** และทำให้ผู้เล่นได้รับวัตถุดิบสำหรับปรุงยาแก้ทางอย่างต่อเนื่อง

3. อัลกอริทึมคำนวณวิถีโค้งพาราโบลาและการกระจายความเสียหายวงกว้าง (Parabolic Trajectory & Area of Effect):

- การเคลื่อนที่วิถีโค้ง: คำนวณความสูงตามสมการควอดราติก **h(t) =** 4 * **H_max * t * (**1 - **t)** (โดย **t** มีค่าตั้งแต่ 0.0 ถึง 1.0) ทำให้ขวดยาลอยโค้งขึ้นสู่จุดสูงสุดกลางอากาศอย่างนุ่มนวลและสมจริง

- การกระจายความเสียหาย (**AoE):** เมื่อขวดตกกระทบ จะใช้การคำนวณระยะทางแบบยุคลิด **d =** sqrt**((x**2 - **x**1)^2 + (**y**2 - **y**1)^2) เพื่อตรวจจับศัตรูในรัศมีระเบิด พร้อมคำนวณการลดทอนดาเมจตามระยะห่าง (**Damage Falloff): Damage = BaseDamage * (**1.0 - (**d / Radius) *** 0.35)

4. อัลกอริทึมการสลับเกราะของบอสใหญ่ตามเครื่องจักรสถานะ (Boss Shield State Machine):

บอสใหญ่ Chaos Overlord ใช้ตัวจับเวลาแบบ Delta Time (dt) เพื่อหมุนเวียนเปลี่ยนเกราะธาตุทุก 7.5 วินาที ตามสมการมอดุโล Index = (Index + 1) % 4 (เวียนระหว่าง ไฟ -> น้ำแข็ง -> พิษ -> สายฟ้า) บังคับให้ผู้เล่นต้องปรับเปลี่ยนกลยุทธ์การปรุงยาให้ตรงตามธาตุของบอสตลอดการต่อสู้

5. อัลกอริทึมระบบตัวคูณคอมโบแบบจำกัดเวลา (Time-decayed Combo Multiplier):

ใช้ตัวนับเวลาถอยหลัง 4.0 วินาที เมื่อผู้เล่นกำจัดมอนสเตอร์ได้อย่างต่อเนื่อง ตัวนับเวลาจะถูกรีเซ็ตและเพิ่มระดับ Combo Streak โดยนำไปคำนวณคะแนนโบนัสตามฟังก์ชัน Score = BaseScore * min(5, ComboCount) ซึ่งกระตุ้นให้ผู้เล่นวางแผนการขว้างยาอย่างต่อเนื่องและแม่นยำ

ตัวอย่างโค้ดอัลกอริทึม

```
// 1. อัลกอริทึม Order-Independent Matching ด้วย Frequency Map ใน RecipeBook.java
public boolean matches(List<IngredientType> ingredients) {
    if (ingredients == null || ingredients.size() != required.size()) return
false;
    Map<IngredientType, Integer> inCounts = countIngredients(ingredients);
    Map<IngredientType, Integer> reqCounts = countIngredients(required);
    return inCounts.equals(reqCounts); // เปรียบเทียบความถี่วัตถุดิบ O(1)
}

// 2. อัลกอริทึมคำนวณพิกัดวิถีโค้งพาราโบลาใน Projectile.java
currentX = startX + (targetX - startX) * progress;
double linearY = startY + (targetY - startY) * progress;
double arcHeight = 120.0 * Math.sin(progress * Math.PI); // ความสูงโค้งพาราโบลา
currentY = linearY - arcHeight;
```

บทที่ 3 สรุป

ปัญหาและอุปสรรคที่พบระหว่างการพัฒนา

1. ข้อจำกัดด้านเวลาและภาระงานที่ซ้อนทับ (Time Constraints & Workload Balance):

เนื่องจากระยะเวลาในการพัฒนาโปรเจกต์ตรงกับช่วงสัปดาห์การสอบและการส่งภาระงานของหลายรายวิชา ทำให้มีเวลาในการเขียนโค้ดและทดสอบระบบอย่างจำกัด

การแก้ไข: ปรับใช้หลักการบริหารจัดการขอบเขตของระบบ (**Scope Management**) โดยจัดลำดับความสำคัญ (**Prioritization**) มุ่งเน้นการสร้างแกนหลักของระบบตามหลักการ OOP ให้ถูกต้องและทำงานได้จริงก่อน (**Core Gameplay & Engine**) จากนั้นจึงค่อยเพิ่มเติมระบบส่วนเสริมในภายหลัง

2. ข้อจำกัดด้านการจัดทำทรัพยากรภาพกราฟิก (Asset Creation Bottleneck):

เกมแนวแฟนตาซีจำเป็นต้องใช้ทรัพยากรภาพจำนวนมาก (ภาพฉากหลัง, สไปรต์มอนสเตอร์ 5 ชนิด, ขวดยา และไอคอนวัตถุดิบ) ซึ่งหากต้องวาดด้วยตนเองทั้งหมดจะไม่สามารถส่งงานได้ทันตามกำหนด

การแก้ไข: ประยุกต์ใช้เทคโนโลยี **Generative AI** เข้ามาช่วยสร้างภาพกราฟิก 2 มิติ (**2D Game Assets**) เพื่อการศึกษา จากนั้นนำมาประมวลผลต่อ (**Post-processing**) ปรับแต่งขนาด อัตราส่วน และตัดพื้นหลัง เพื่อให้นำเข้าสู่ระบบ **Java Graphics2D** ได้อย่างสมบูรณ์

3. ความซับซ้อนในการเชื่อมโยงโครงสร้างคลาสตามหลักการ OOP:

เนื่องจากโปรเจกต์มีโครงสร้างคลาสมากกว่า **20** คลาส การเชื่อมโยงข้อมูลระหว่างระบบควบคุม (**GameEngine**), หน้าจอแสดงผล (**GameplayPanel**) และข้อมูลตัวละคร (**Entities**) มักเกิดความสับสนและข้อผิดพลาดในการส่งผ่านพารามิเตอร์

การแก้ไข: ศึกษาโครงสร้างเชิงสถาปัตยกรรม (**Architectural Pattern)** และนำหลักการ **Low Coupling** มาปรับใช้ โดยให้ **GameEngine** ทำหน้าที่เป็นศูนย์กลางเพียงจุดเดียว (**Single Controller)** และใช้อินเทอร์เฟซ เช่น **Renderable, Damageable** เพื่อลดการผูกมัดของคลาส

4. ปัญหาการคำนวณตำแหน่งและการแสดงผลบนหน้าจอต่างขนาด (Screen Scaling Issue):

เมื่อทำการทดสอบการเล่นบนหน้าจอคอมพิวเตอร์ที่มีความละเอียดและขนาดหน้าต่างที่แตกต่างกัน พบปัญหาพิกัดการคลิกเมาส์ไม่ตรงกับภาพกราฟิกบนหน้าจอ

การแก้ไข: ออกแบบและพัฒนาคลาส **ScaleHelper** เพื่อสร้างระบบพิกัดเสมือน (**Virtual Coordinate System: 1280x720)** ทำหน้าที่แปลงพิกัดจริงของหน้าจอให้สอดคล้องกับพิกัดในเกมอัตโนมัติ ทำให้เล่นได้ทุกขนาดหน้าจอโดยตำแหน่งไม่คลาดเคลื่อน

จุดเด่นของโปรแกรมที่ไม่เหมือนใคร

1. สถาปัตยกรรมโปรแกรมถูกต้องตามหลักการ OOP ระดับสูง:

มีการประยุกต์ใช้แนวคิดเชิงวัตถุอย่างครบถ้วน ทั้ง **Abstract Class, Interface (Renderable, Damageable), Polymorphism (**การระเบิดและการส่งผลของขวดยาแต่ละชนิด) และ **Encapsulation** โค้ดถูกจัดหมวดหมู่อย่างเป็นระเบียบและง่ายต่อการต่อยอด

2. ระบบเกมเพลย์ผสมผสานการวางแผนและความเร็ว (Real-time Alchemy Defense):

นำเสนอลูปการเล่นที่ไม่ซ้ำใคร โดยผู้เล่นต้องจำสูตรและผสมยา **3** วัตถุดิบแบบไม่จำกัดลำดับ (**Order-Independent)** เพื่อแก้ทางธาตุของมอนสเตอร์ที่เดินเข้ามาแบบ **Real-time**

3. ระบบช่วยเหลืออัจฉริยะในเกม (Smart Player Assistance):

มีระบบสายพานสุ่มวัตถุดิบตามภัยคุกคาม (**Smart Spawning)** ที่คอยช่วยวิเคราะห์และเร่งสุ่มของแก้ทางให้ผู้เล่น **85%** พร้อมทั้งระบบคาถาชี้แนะสูตรยาเหนือหม้อต้ม ช่วยลดความสับสนของผู้เล่น

4. กราฟิกและเอฟเฟกต์ภาพแฟนตาซี 2 **มิติ:**

ผสมผสานภาพกราฟิกและเอฟเฟกต์แอนิเมชันเชิงฟิสิกส์ได้อย่างลงตัว เช่น การลอยของขวดยา วิถีโค้งพาราโบลา, วงแหวนล็อกเป้าหมาย, และละอองเวทมนตร์ตามธาตุ

5. ระบบเสียงสังเคราะห์แท้ในตัว (Built-in Audio Synthesizer):

พัฒนาระบบเสียงดนตรีประกอบและเสียงเวทมนตร์ด้วยโค้ด **Java** แท้ๆ (**Sound Synthesis)** ทำให้เกมมีเสียงครบถ้วนโดยไม่ต้องพึ่งพาไฟล์เสียงขนาดใหญ่จากภายนอก เสาหลักมาประยุกต์ใช้จริงอย่างสมบูรณ์แบบ โค้ดสะอาดและเป็นระเบียบ

คำแนะนำสำหรับผู้สอนที่อยากให้อธิบาย หรือที่เรียนแล้วไม่เข้าใจ หรืออยากให้เพิ่มสำหรับน้อง ๆ รุ่นต่อไป

ส่วนตัวผมไม่ค่อยถนัดการเขียนโค้ดเท่าไหร่ แต่จะชอบการออกแบบระบบ การมองภาพรวม และการดู Flow การทำงานต่าง ๆ ของระบบมากกว่า หลายครั้งเวลาที่ต้องสอบควิซหรือทำแล็บ แล้วทำไม่ได้ติดต่อกัน ก็แอบรู้สึกท้อแท้ และพาลทำให้มองข้ามความสามารถด้านอื่น ๆ ในตัวเองไป เพราะหากจะให้ลงมือเขียนโค้ดเองหรือทำทุกอย่างด้วยตัวเองทั้งหมด ก็อาจจะเกินกำลังและขอบเขตความสามารถของผมในตอนนี้

การพยายามฝืนทำในสิ่งที่ไม่ถนัด โดยไม่มีตัวช่วยหรือไม่ได้ฝึกใช้เครื่องมืออื่น ๆ มาเสริม อาจทำให้เราเหนื่อยล้าจนเกินไป และหมดกำลังใจในการเรียนได้ง่าย ๆ (แต่ผมไม่ได้หมายความว่าการพยายามฝึกฝนด้วยตัวเองเป็นเรื่องไม่ดีนะครับ)

อยากฝากถึงน้อง ๆ รุ่นต่อ ๆ ไปว่า เรากำลังเรียนเขียนโค้ดในยุคที่เทคโนโลยีและ AI เก่งขึ้นมาก ๆ อยากให้น้อง ๆ ลองศึกษาและฝึกใช้งานมัน เพื่อเป็นเครื่องมือช่วยในการเรียนและการทำงานของเราให้ดีและมีประสิทธิภาพยิ่งขึ้น

การที่เราสอบไม่ได้ หรือเรียนในห้องไม่เข้าใจในตอนนี้ ไม่ได้หมายความว่าเราจะไม่ประสบความสำเร็จ โลกภายนอกรั้วมหาวิทยาลัยนั้นกว้างใหญ่มาก ๆ ตอนนี้พี่เองก็มีช่วงที่ท้อใจมาก ๆ เหมือนกันเพราะข้อสอบยากเหลือเกิน แต่อยากให้ทุกคนใส่ใจสุขภาพกายและสุขภาพใจของตัวเองเป็นอันดับแรก เพราะร่างกายและจิตใจของเรามีค่าที่สุด อย่าปล่อยให้ปัญหาหรือความล้มเหลวชั่วคราวมาทำให้ชีวิตเราเดินต่อไม่ได้

เราทุกคนล้มได้ พลาดได้ แต่ต้องมองไปข้างหน้าเสมอ สิ่งที่ผิดพลาดในวันนี้ อย่ามัวแต่เก็บมาโทษตัวเองซ้ำไปซ้ำมาจนไม่มีความสุขในการใช้ชีวิต ขอแค่เก็บมันไว้เป็นบทเรียน อะไรที่เกิดขึ้นแล้วเราแก้ไขอดีตไม่ได้ ปล่อยให้มันผ่านไป แล้วพยายามทำครั้งหน้าให้ดีขึ้นเรื่อย ๆ ตราบใดที่เรายังไม่ตาย ชีวิตนี้ก็ยังไม่จบ สู้ต่อไปครับ

ขอบคุณที่สละเวลาอ่านนะครับ ขอบคุณครับ